

Default Build Matrix function:

The current implementation of `build_traffic_matrix()` is a placeholder designed only to enable the Timeloop to BookSim pipeline to be able to run. It does not derive communication from the computation or mapping generated by Timeloop. Assumes a fixed communication pattern where 60% of traffic is exchanged between neighbouring tiles and 40% is directed to a single memory-controller (assumed node 0).

The model uses only the aggregate DRAM access count reported by Timeloop. It completely ignores which tensor is being accessed by which tile and how the computation is spatially distributed across tiles. This doesn't match how our neural network works or how our workload runs or what operations are present in our workload.

The final build matrix function should be able to do the following:

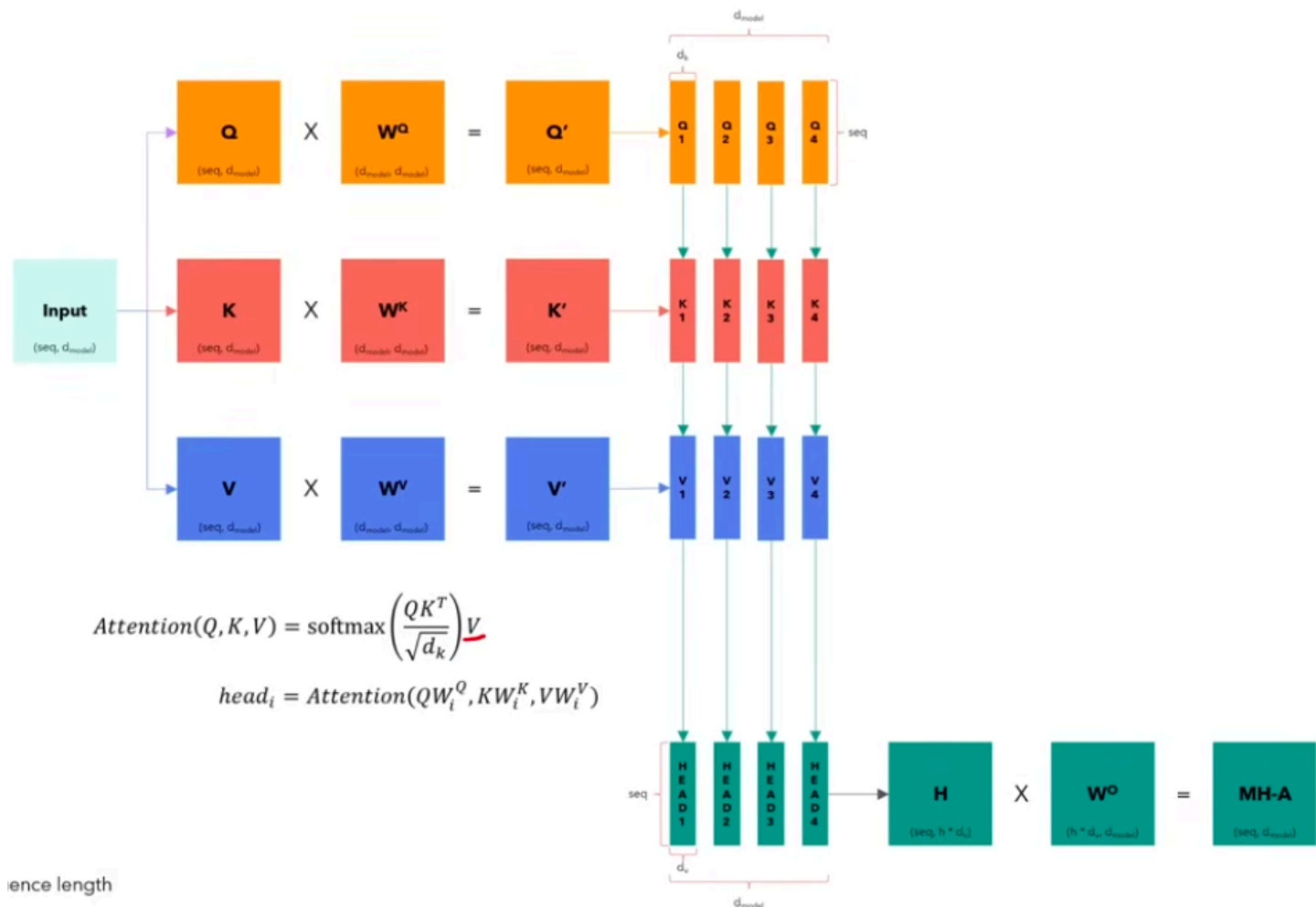
- Get the total number of dram accesses per operation
- Have an understanding of which tiles are accessing this data (we will do this by making sure the problem.yaml is per tile specific), in case there is data sharing between tiles it should be aware of that.
- Finally map accesses for each tile in two categories, Off chip (Dram to tile) and inter tile (for data sharing when needed between tiles)

Attention Mapping Spec:

Mapping: 16 tiles, 32 heads - 2 full heads per tile (**chosen for least inter tile communication**)

keeping full heads per tile confines all attention math to local computation and pushes the only unavoidable communication to one place (the output projection)

So we compute attention locally for all heads in their specific tiles till we get the head outputs. The next step is to concatenate the outputs of each head to get the original tensor size and then do a matmul with W_o (output weights). Instead of doing this we won't move the head tensors anywhere, instead we will shard the output matrix into rows that will match the size of each head then send these to the tiles. The output of this operation will be of the correct output that we were initially getting but will be a fraction of it, so we need to add up all the output matrices after doing this step to get the correct output which would've resulted from the standard procedure of concating and then matmul with W_o . This is the first step where we will need data from multiple tiles.

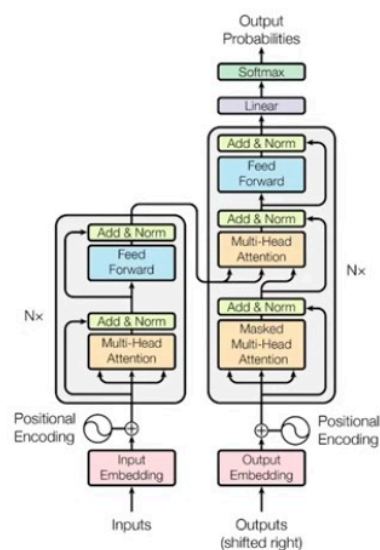


Tensor	Local or transferred	When
X (input slice)	local	—
W_q, W_k, W_v , per-tile heads	local	—
Q, K, V	local	—
Attention scores, softmax output	local	—
Partial output-projection result	transferred	after W_o matmul
Partial FFN down_proj result (if TP)	transferred	after down_proj

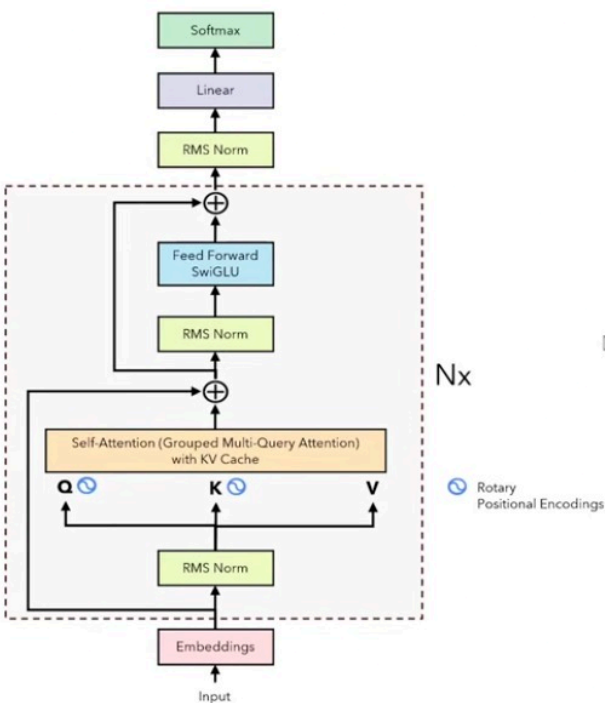
LLaMA 2-7B

Model:

Transformer vs LLaMA



Transformer
("Attention is all you need")



LLaMA

Tensor Sizes:

Step / File	Mathematical Operator	Input A	Input B	Output
q_proj.yaml	$Q = XW_q$	2048×4096	4096×4096	2048×4096
k_proj.yaml	$K = XW_k$	2048×4096	4096×4096	2048×4096
v_proj.yaml	$V = XW_v$	2048×4096	4096×4096	2048×4096
qk.yaml	$\text{Scores} = QKT$	2048×128	128×2048	2048×2048
sv.yaml	$\text{Context} = \text{Scores} \cdot V$	2048×2048	2048×128	2048×128
out_proj.yaml	$\text{Output} = \text{Context} \cdot W_o$	2048×4096	4096×4096	2048×4096
gate_proj.yaml	$\text{Gate} = XW_{\text{gate}}$	2048×4096	4096×11008	2048×11008
up_proj.yaml	$U_p = XW_{\text{up}}$	2048×4096	4096×11008	2048×11008
down_proj.yaml	$\text{Down} = (\text{Gate} \cdot U_p)W_{\text{down}}$	2048×11008	11008×4096	2048×4096

File	Shape	H needed?	Row- or column-parallel
qkv_proj.yaml	M=2048, K=4096, N=768	no	column
attention_qk.yaml	B=1, H=2, M=2048, N=2048, D=128	yes (local heads)	—
attention_sv.yaml	same	yes	—
out_proj.yaml	M=2048, K=256, N=4096	no	row (→ all-reduce)
gate_up_proj.yaml	M=2048, K=4096, N=1376 (688×2)	no	column
down_proj.yaml	M=2048, K=688, N=4096	no	row (→ all-reduce)